

Haneen Alaa Hassan

Depi day5 c#

Essay Questions

1. What is the purpose of the finally block?

The finally block is used to execute code regardless of whether an exception occurs or not. It is commonly used for cleanup operations such as closing files, database connections, or releasing resources.

2. How does `int.TryParse()` improve program robustness compared to `int.Parse()`?

`int.TryParse()` is safer because it does not throw an exception when the input is invalid. Instead, it returns false if the conversion fails. `int.Parse()` throws a `FormatException` when the input is not a valid integer.

3. What exception occurs when trying to access Value on a null `Nullable<T>`?

Accessing the Value property of a nullable value that contains null throws an `InvalidOperationException`.

4. Why is it necessary to check array bounds before accessing elements?

Array indexes must be within the valid range of the array. Accessing an invalid index causes an `IndexOutOfRangeException`. Checking the bounds prevents this exception and makes the program safer.

5. How is `GetLength(dimension)` used in multi-dimensional arrays?

`GetLength(dimension)` returns the number of elements in a specific dimension of a multi-dimensional array.

For example:

`array.GetLength(0)`

returns the number of rows, while:

```
array.GetLength(1)
```

returns the number of columns in a two-dimensional array.

6. How does memory allocation differ between jagged arrays and rectangular arrays?

A rectangular array is stored as one multi-dimensional array with a fixed number of elements in each dimension. A jagged array is an array of arrays, where each row is a separate array and can have a different size.

Therefore, jagged arrays provide more flexibility but may have additional memory overhead because each row is a separate array.

7. What is the purpose of nullable reference types in C#?

Nullable reference types help developers identify possible null values during compile time. They provide warnings when a reference that may be null is used without proper checking, which helps reduce `NullReferenceException` errors.

8. What is the performance impact of boxing and unboxing in C#?

Boxing and unboxing can reduce performance because boxing converts a value type into an object and usually requires an allocation on the heap. Unboxing requires converting the object back to its original value type. Frequent boxing and unboxing can increase memory usage and decrease performance.

9. Why must out parameters be initialized inside the method?

An out parameter does not need to be initialized before calling the method. However, the method must assign a value to every out parameter before it returns. This guarantees that the caller receives a valid value.

10. Why must optional parameters always appear at the end of a method's parameter list?

Optional parameters must appear at the end because parameters without default values must be provided by the caller. If an optional parameter appeared before a required parameter, it could create ambiguity when passing arguments.

11. How does the null propagation operator ?. prevent NullPointerException?

The null propagation operator ?. checks whether an object is null before accessing its member. If the object is null, the expression returns null instead of trying to access the member and throwing a NullPointerException.

12. When is a switch expression preferred over a traditional if statement?

A switch expression is preferred when a single value needs to be compared against multiple possible patterns or values. It makes the code shorter, cleaner, and easier to read compared to multiple if-else statements.

13. What are the limitations of the params keyword in method definitions?

The params parameter must be the last parameter in the method. A method can have only one params parameter, and it must be a single-dimensional array type. The caller can pass individual values or an array of the specified type.